

Reviewer Pack — Minimal Root Count of Integer Polynomials and Resolution Pro...

Entry 21c70b173bfc · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 14:16:36 UTC

Minimal Root Count of Integer Polynomials and Resolution Proof Width

Entry ID: 21c70b173bfc **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 14:16:36 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Root Theory) **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every CNF formula φ with n variables, the minimal number of distinct roots of the polynomial formed by the determinant of the matrix representing the incidence relation of literals in φ is linearly correlated with its resolution proof width $w(\varphi)$, such that the minimal root count $r(\min) \leq cw(\varphi)$ for some constant c .

Rationale (proposer's reasoning):

Root theory provides a way to analyze the structure of polynomials, which could potentially expose underlying complexities in the logical structure of CNF formulas. A direct connection between this algebraic invariant and proof complexity might reveal new insights into the nature of computational difficulty.

Taxonomy category: resolution-proof-complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a301b96835fd022c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all 30 random seeds, the ratio of the minimal root count $r(\min)$ to the resolution proof width $w(\varphi)$ is greater than or equal to a constant c ($r(\min) / w(\varphi) \geq c$), and there are no seeds where $r(\min) > cw(\varphi)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'algebraic geometry' AND 'resolution proof complexity' AND integer polynomials' - 'determinant of incidence matrix' AND 'root theory' AND resolution proof width-minimal root count integer polynomials CNF formula resolution proof width

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2 * n):
            clause = [random.randint(-n, -1), random.randint(1, n)]
            clauses.append(clause)
        return clauses

    def incidence_matrix(cnf, n):
        m = len(cnf)
        M = [[0] * (2 * n) for _ in range(m)]
        for i, clause in enumerate(cnf):
            for lit in clause:
                if lit > 0:
                    M[i][lit - 1] = 1
                else:
                    M[i][-lit - 1] = 1
        return M

    def determinant(M):
        n = len(M)
        if n == 1:
            return M[0][0]
        det = 0
        for j in range(n):
            submatrix = [row[:j] + row[j+1:] for row in M[1:]]
            sign = (-1) ** (1 + j)
            det += sign * M[0][j] * determinant(submatrix)
```

```

    return det

def roots_count(poly, n):
    if len(poly) == 1:
        return 0
    if poly[-1] != 0:
        return 1 + roots_count([poly[i] / poly[-1] for i in range(len(poly)-1)], n-1)
    else:
        return roots_count(poly[:-1], n-1)

def resolution_width(cnf):
    # Simplified resolution width calculation
    return len(cnf)

n = random.randint(5, 40)
cnf = generate_cnf(n)
M = incidence_matrix(cnf, n)
det_poly = determinant(M)
r_min = roots_count(det_poly, n)
w_phi = resolution_width(cnf)

if w_phi == 0:
    return {
        "metric_name": "r(min) / w( $\phi$ )",
        "metric_value": None,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "w( $\phi$ ) is zero"
    }

ratio = r_min / w_phi
return {
    "metric_name": "r(min) / w( $\phi$ )",
    "metric_value": ratio,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": ratio >= 0.5, # Placeholder constant c=0.5
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2*i + 3 for i in range(5, 8)]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results if r["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value:.2f} std={std_value:.2f} support_fraction={support_fraction:.2f}")

```

```

elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"r(min) > cw(φ)\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE no seeds supported the conjecture")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete its execution to verify the conjecture. | next: Re-run the test with increased time limits or optimize the code to ensure it completes within the given time frame.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	23949
2	preregistration	ollama_remote	glm4:latest	0	0	11057
3	novelty	ollama_remote	glm4:latest	0	0	19155
4	novelty	ollama_remote	glm4:latest	0	0	26899
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16898
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13676
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17197
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14523
9	judge	ollama_remote	glm4:latest	0	0	13465

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 156819 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/21c70b173bfc.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/21c70b173bfc.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/21c70b173bfc.tar.gz` (if generated)